Maximize Expression

AlgoExpert · Hard · Dynamic Programming

The problem

- Max of a - b + c - d over indices a < b < c < d (0 if fewer than 4 elements).
- Four rolling DP layers.

Intuition

- Build the expression one term at a time: best a, then best a minus b, and so on, each over a prefix.

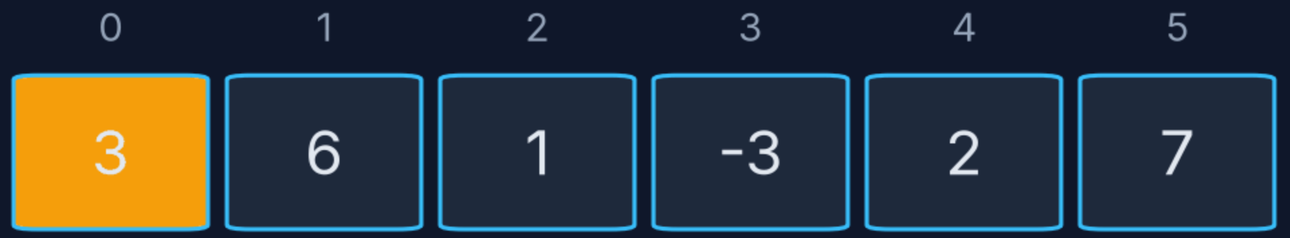
Brute force vs optimal

- Brute force: all $i<j<k<l$ → $O(n^4)$
- Four DP arrays → $O(n)$
- Four running scalars → $O(1)$ space

Approach

- 1. Four arrays of running bests
- 2. Each layer uses the previous layer up to the index before
- 3. Return the last layer's max

Worked example

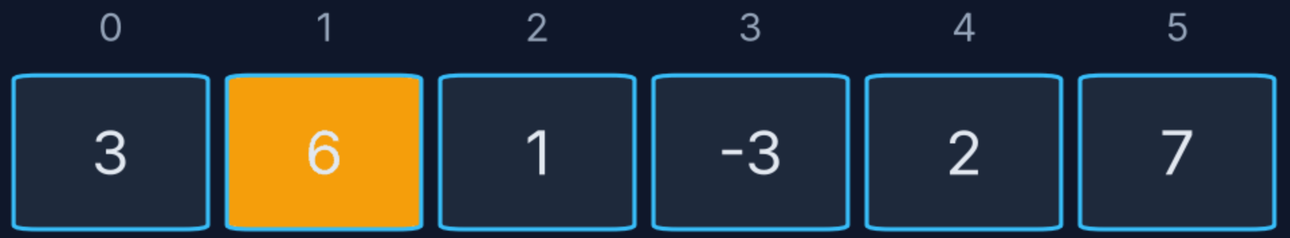


max a max a−b max a−b+c a−b+c−d

3	–	–	–

i=0: best a−b+c−d = –

Worked example

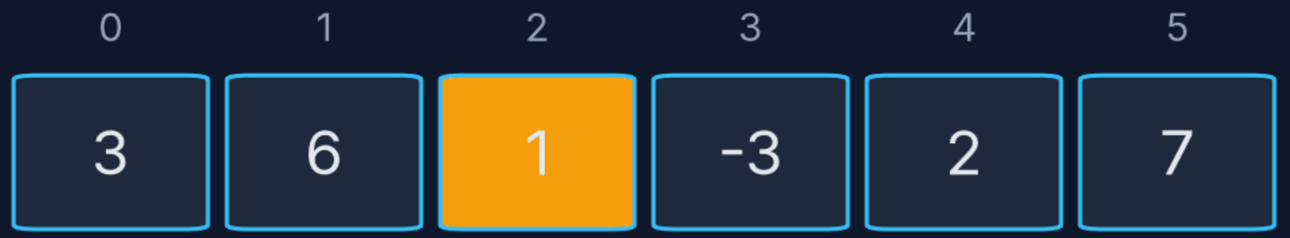


max a max a − b max a − b + c a − b + c − d

6	-3	–	–

i=1: best a − b + c − d = –

Worked example

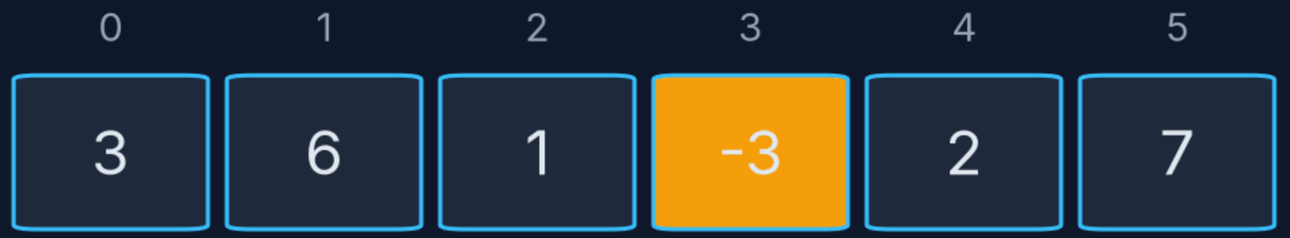


max a max a − b max a − b + c a − b + c − d

6	5	-2	–

i=2: best a − b + c − d = –

Worked example

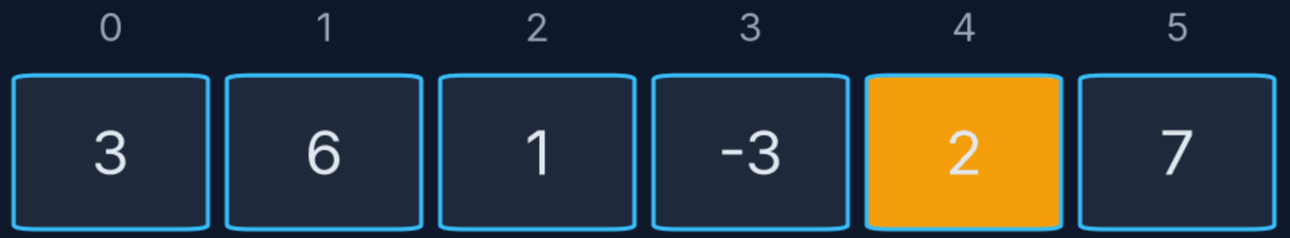


max a max a−b max a−b+c a−b+c−d

6	9	2	1

i=3: best a−b+c−d = 1

Worked example

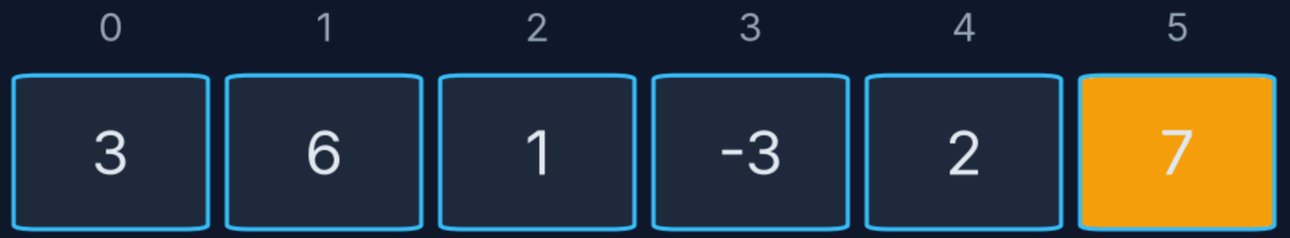


max a max a − b max a − b + c a − b + c − d

6	9	11	1

i=4: best a−b+c−d = 1

Worked example



max a max a−b max a−b+c a−b+c−d

7	9	16	4

i=5: best a−b+c−d = 4

Java solution

```java
public class MaximizeExpression {
    public static int maximizeExpression(int[] a) {
        if (a.length < 4) return 0;
        long s1 = Long.MIN_VALUE / 4, s2 = Long.MIN_VALUE / 4, s3 = Long.MIN_VALUE / 4, s4 = Long.MIN_VALUE / 4;
        for (int i = 0; i < a.length; i++) {
            if (i >= 3) s4 = Math.max(s4, s3 - a[i]);
            if (i >= 2) s3 = Math.max(s3, s2 + a[i]);
            if (i >= 1) s2 = Math.max(s2, s1 - a[i]);
            s1 = Math.max(s1, a[i]);
        }
        return (int) s4;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Fewer than 4 elements → 0
- Update later stages before earlier ones
- Avoid overflow with sentinels

Interview tips & follow-ups

- Explain the staged DP

Recap

- Build the expression one term at a time: best a, then best a minus b, and so on, each over a prefix.
- Complexity: O(n)